



Escuela de Ingeniería y Ciencias
Ingeniería en Ciencia de Datos y Matemáticas
Bloque MA2006B Uso de Álgebras Modernas para Seguridad y Criptografía

Reporte Técnico

Gestor de Identidades, Gestión y Verificación de Documentos

Equipo 3 — Grupo 601

Estudiante	Matrícula
Daniel Eugenio González Limas	A01285898
Sebastián Zaragoza Díaz	A01286211
Cedrick Patricio Treviño Ortiz	A01198868
Jorge Eduardo González Cantú	A01571510
André Robles Bueckmann	A01706832

Profesores: Raúl Gómez, Daniel Otero y Alberto F. Martínez

Socio Formador: Casa Monarca. Ayuda Humanitaria al Migrante, A.B.P.
Monterrey, Nuevo León. 14 de junio de 2026.

Índice de Contenidos

Índice de Contenidos	2
Índice de Tablas	2
Índice de Figuras	2
Abstract	3
Introducción	3
Marco de referencia	4
Métodos de autenticación revisados	4
Conceptos criptográficos de clave pública e integridad	4
Estándares retomados en la solución	5
Desarrollo	5
Propuesta de solución y justificación	5
Desarrollo de componentes — Backend (Access-Control-CM-API)	7
Desarrollo de componentes — Frontend (Access-Control-CM-Web)	7
Conexión de los componentes	8
Gestión y verificación de documentos	9
Resultados	10
Resultado del Backend	10
Resultado del Frontend	11
Resultado de la solución completa	11
Casos de uso y validaciones	12
Conclusiones y trabajo futuro	13
Referencias	14
Anexos	14

Índice de Tablas

- Tabla 1. Métodos de autenticación revisados.
- Tabla 2. Matriz de permisos por rol (RBAC).
- Tabla 3. Inventario de algoritmos criptográficos empleados.
- Tabla 4. Endpoints principales de la API por rol.
- Tabla 5. Casos de uso y validaciones.

Índice de Figuras

- Fig. 1. Diagrama de contenedores (arquitectura de mismo origen).
- Fig. 2. Secuencia de inicio de sesión con doble factor (TOTP).
- Fig. 3. Flujo de firma digital con WebAuthn/FIDO2.
- Fig. 4. Diagrama entidad-relación de la base de datos.
- Fig. 5. Vistas reales del sistema por rol (capturas).

Abstract

(este es un primer borrador.)

Casa Monarca, casa de apoyo a personas migrantes y refugiadas, gestiona información personal y documentos legales altamente sensibles cuyo manejo manual representa un riesgo de confidencialidad, integridad y trazabilidad. La solución se desarrolló en dos etapas complementarias: la **Etap 1 (Gestor de identidades)** construyó el control de identidad y acceso, registro por invitación verificada, autenticación de doble factor (TOTP y passkeys WebAuthn/FIDO2) y control de acceso basado en roles (RBAC) de cuatro niveles, y la **Etap 2 (Gestión y Verificación de Documentos)** añadió el manejo seguro del ciclo de vida documental, versionado con hash SHA-256, firma digital criptográfica con no-repudio, verificación de documentos (incluida la verificación local en el navegador) y una bitácora de auditoría completa. La solución se construyó como dos aplicaciones de mismo origen, un frontend React/TypeScript y un backend Laravel/PHP con MySQL, bajo una metodología DevSecOps y orientada a un hosting compartido tipo HostGator. El resultado es un sistema funcional en ambiente de staging que garantiza confidencialidad, integridad, autenticidad, disponibilidad y no-repudio de la información, ofreciendo de esta manera, una solución criptográficamente fuerte, amigable, y confiable para Casa Monarca.

Introducción

Casa Monarca. Ayuda Humanitaria al Migrante es una organización de la sociedad civil que brinda atención a personas migrantes y refugiadas. En su operación diaria procesa datos personales y documentos legales extremadamente sensibles (identificaciones, expedientes, documentos oficiales), pertenecientes a una población en situación de alta vulnerabilidad. El manejo de esta información mediante procesos manuales y archivos no centralizados incrementa el riesgo de fugas, pérdidas, alteraciones y falta de trazabilidad sobre quién accede o modifica cada documento.

El reto se desarrolló en dos etapas complementarias. La Etapa 1 (Gestor de Identidades) se centró en controlar la identidad y el acceso del personal: alta por invitación verificada, autenticación de doble factor y control de acceso basado en roles de cuatro niveles. La Etapa 2 (Gestión y Verificación de Documentos) construyó sobre esa base el manejo seguro del ciclo de vida documental: carga, versionado con integridad por hash, firma digital con no-repudio y verificación de documentos. En conjunto, el objetivo es garantizar los cinco principios de la seguridad de la información: confidencialidad, integridad, autenticidad, disponibilidad y no-repudio, cumpliendo con los Derechos ARCO y la Ley Federal de Protección de Datos Personales en Posesión de los Particulares, aplicando algoritmos de clave pública y funciones resumen (hash) para asegurar la base de datos, las firmas y la verificación.

Este reporte responde a la pregunta ¿qué hicimos?: describe la propuesta de solución, las decisiones de diseño, los componentes de backend y frontend desarrollados, su conexión, y los resultados obtenidos. El

detalle de ¿cómo lo hicimos? Se extiende en la documentación del desarrollador (manual de desarrollo / SDK).

Marco de referencia

La gestión de identidades y la firma electrónica son problemas resueltos por múltiples tecnologías y estándares. A continuación se documentan las soluciones y conceptos revisados, y se indica qué ideas se retomaron para la solución propia.

Métodos de autenticación revisados

Los métodos de autenticación determinan cómo un colaborador acredita su identidad al acceder a la plataforma.

Tabla 1. *Métodos de autenticación revisados (elaboración propia). Los métodos de autenticación determinan cómo un colaborador acredita su identidad al acceder a la plataforma. A continuación se describen los cuatro métodos identificados.*

Método	Explicación	Ejemplo
Autenticación de dos factores (2FA)	Requiere dos medios de identificación distintos (algo que se sabe + algo que se tiene/es) para reforzar la seguridad de la cuenta.	Mitec, Facebook, AWS.
Código de un solo uso (OTP)	Código numérico/alfanumérico de validez temporal enviado o generado para validar una operación.	BBVA, Amazon, Gmail.
Certificado digital (eFirma)	Archivo electrónico que acredita la autenticidad del usuario usando criptografía de clave pública (PKI).	e.firma del SAT para declarar/facturar.
Token (físico o digital)	Dispositivo o app que genera claves dinámicas locales sincronizadas con el servidor, sin requerir red.	Token físico bancario; token digital BBVA/Citibanamex/HSB C.

Conceptos criptográficos de clave pública e integridad

El bloque aborda criptografía de clave pública y funciones resumen. Como referencia de implementación se revisaron bibliotecas y sistemas existentes: Pycryptodome como biblioteca de Python para clave pública, y el sistema del SAT como caso real de administración de identidades de contribuyentes mediante certificados (e.firma).

La aplicación terminó por tener el algoritmo “RSASSA-PKCS1-v1_5-SHA256”, el cuál es utilizado para firma digital en los documentos que gestiona la aplicación. Incluye RSA, el algoritmo criptográfico de cifrado público; SSA, que refiere más específicamente al esquema de la firma digital; PKCS versión 1.5, que es el formato con el que se construye la firma (incluyendo un concepto llamado padding); por último, SHA-256 que no es más que la función hash utilizada antes de firmar. (RFC 8017: PKCS #1: RSA Cryptography Specifications, n.d.).

El algoritmo RSA es de encriptación asimétrica, y fue uno de los protocolos vistos durante el curso de Algebras Modernas y Criptografía del ITESM, en 2026.

Estándares retomados en la solución

De la revisión anterior, la solución adopta estándares abiertos y ampliamente auditados en lugar de criptografía propia:

- **TOTP (RFC 6238/4226)** para el segundo factor basado en tiempo.
- **WebAuthn / FIDO2** para passkeys y firma digital con no-repudio.
- **AES-256-CBC** (cifrado simétrico) y **SHA-256 / HMAC-SHA256** para confidencialidad e integridad de datos sensibles.
- **RSASSA-PKCS1-v1_5** (Firma digital de documentos). Sostenido por función de hash SHA-256.
- **YubiKey** como dispositivo físico de autenticación utilizado en conjunto con el protocolo FIDO2.

Desarrollo

Esta sección documenta qué se desarrolló: la propuesta, su justificación y los componentes construidos.

Propuesta de solución y justificación

Se desarrolló Casa Monarca Access Control, un sistema de gestión de documentos confidenciales con firma digital, control de acceso basado en roles (RBAC), auditoría completa y de gestión y verificación de documentos. El sistema permite al personal autorizado: registrarse por invitación verificada, autenticarse con doble factor, subir y versionar documentos sensibles, firmarlos digitalmente con passkeys, gestionar flujos de aprobación por roles y dejar un rastro auditable de todas las acciones.

La solución se estructura en dos aplicaciones desplegadas bajo el mismo origen:

- Un **frontend web** (SPA en React + TypeScript).
- Un **backend API REST** (Laravel/PHP) con base de datos MySQL.

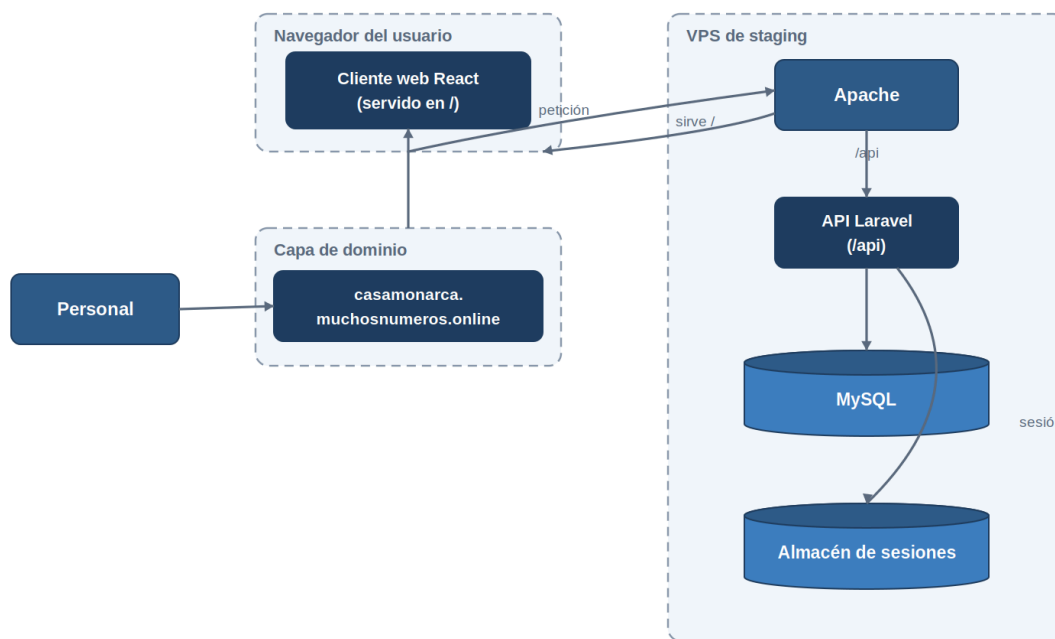


Fig. 1. Diagrama de contenedores: arquitectura de mismo origen — Apache sirve el frontend en / y hace proxy a la API en /api (elaboración propia).

Decisiones de diseño y su justificación:

- **Arquitectura de mismo origen.** Servir frontend y API bajo el mismo dominio simplifica CORS y permite usar cookies de sesión seguras (HttpOnly) en lugar de guardar tokens en el navegador, reduciendo la superficie de ataque (robo de tokens por XSS).
- **Hosting compartido tipo HostGator en lugar de la nube.** Se prioriza un modelo de hosting económico y sostenible para una organización sin fines de lucro, que el propio personal pueda mantener. Esto impone restricciones (PHP/MySQL, sin servicios administrados) que condicionaron el stack y el despliegue.
- **Sesiones server-side + CSRF en lugar de JWT en cliente.** Mantener la sesión del lado del servidor evita exponer credenciales de larga vida en el navegador; las operaciones sensibles (POST/PUT/DELETE) exigen un token CSRF.
- **Criptografía estándar y auditada, no propia.** Se usan Web Crypto API (cliente), OpenSSL y el Encrypter de Laravel (servidor), evitando implementar primitivas propias.
- **Firma con passkeys (WebAuthn/FIDO2).** La clave privada nunca sale del dispositivo del usuario, garantizando no-repudio.

Nota de diseño vs. implementación. La documentación conceptual inicial describía un esquema con certificados X.509, mTLS y una CA interna tipo e.firma. En la implementación real, la identidad fuerte y la firma se resolvieron con WebAuthn/FIDO2 (passkeys) + TOTP, por su mejor encaje con navegadores y hosting compartido. Esta diferencia se documenta de forma transparente.

Desarrollo de componentes — Backend (Access-Control-CM-API)

- **Stack:** PHP 8.3 + Laravel, MySQL 8 con el ORM Eloquent.
- **Arquitectura por capas:** Rutas (routes/api.php) → Controllers (Api/{Auth, Admin, Audit, Documents}) → Services → Models (Eloquent) → Base de datos.
- **Autenticación:** sesiones server-side + cookies + CSRF. Contraseñas hasheadas con Argon2id/bcrypt; segundo factor TOTP y/o passkeys.
- **Roles (RBAC):** admin, coordinator, non_coordinator, volunteer, aplicados con middleware (requireRole, requireSecurityEnrollment).
- **Gestión documental:** documentos, revisiones versionadas (cadena padre→hijo con hash SHA-256), firmas y lápidas (tombstones) de borrado.
- **Auditoría:** bitácora append-only (audit_events) con ~49 tipos de evento.

El control de acceso por rol define qué módulos y acciones puede ejecutar cada usuario.

Tabla 2. Matriz de permisos por rol (RBAC) (elaboración propia). El sistema implementa un modelo de control de acceso basado en roles (Role-Based Access Control), en el que cada perfil de usuario cuenta con un conjunto específico de permisos sobre las acciones disponibles en la plataforma. Se definen cuatro roles con niveles de privilegio progresivos.

Acción	Voluntario	No-coord.	Coordinador	Admin
Subir documento	✓	✓	✓	✓
Ver / descargar / verificar documentos	—	✓	✓	✓
Subir nueva revisión	—	—	✓	✓
Firmar revisión	—	—	✓	✓
Crear / gestionar invitaciones	—	—	✓	✓
Aprobar / rechazar documentos	—	—	—	✓
Gestionar usuarios, roles y recuperación	—	—	—	✓
Borrar documento (deja tombstone)	—	—	—	✓
Auditoría y libro de firmas	—	—	—	✓

Requisitos de enrolamiento de seguridad antes de operar: coordinador requiere TOTP + passkey; no-coordinador y voluntario requieren TOTP. Las acciones sensibles (firmar, cambiar rol, borrar) exigen además una passkey assertion fresca.

Desarrollo de componentes — Frontend (Access-Control-CM-Web)

- **Stack:** React 19 + TypeScript + Vite (SPA, build a archivos estáticos).
- **Comunicación con la API:** capa lib/api.ts vía fetch con credentials: include y token CSRF.
- **Criptografía de cliente:** Web Crypto API nativa, solo para verificar firmas (nunca para crear claves privadas).

- **Rutas protegidas por rol:** /app (todos los roles), /app/admin (solo admin), /403 (acceso denegado).
- **Módulos por rol:** subida de documentos, revisión/VCS de documentos, invitaciones, panel de administración, página de auditoría y libro de firmas (signing ledger).
- **Enrolamiento 2FA:** generación de QR para TOTP (qrcode) y registro de passkeys.

Conexión de los componentes

- **Entorno local:** el frontend corre en `http://localhost:5173` y la API en `http://127.0.0.1:8000`; Vite hace proxy de `/api/*` hacia Laravel para evitar problemas de CORS. Laravel se conecta a MySQL local (puerto 3306) con credenciales del archivo `.env`.
- **Despliegue:** Apache actúa como borde de mismo origen, sirve los archivos estáticos de React en `/` y hace proxy a la API en `/api`; Laravel procesa la solicitud y lee/escribe en MySQL. El frontend se sube como artefacto estático y la API se actualiza con `git pull remoto` mediante `scripts/deploy-staging.sh`.

El siguiente diagrama muestra el flujo de inicio de sesión con segundo factor.

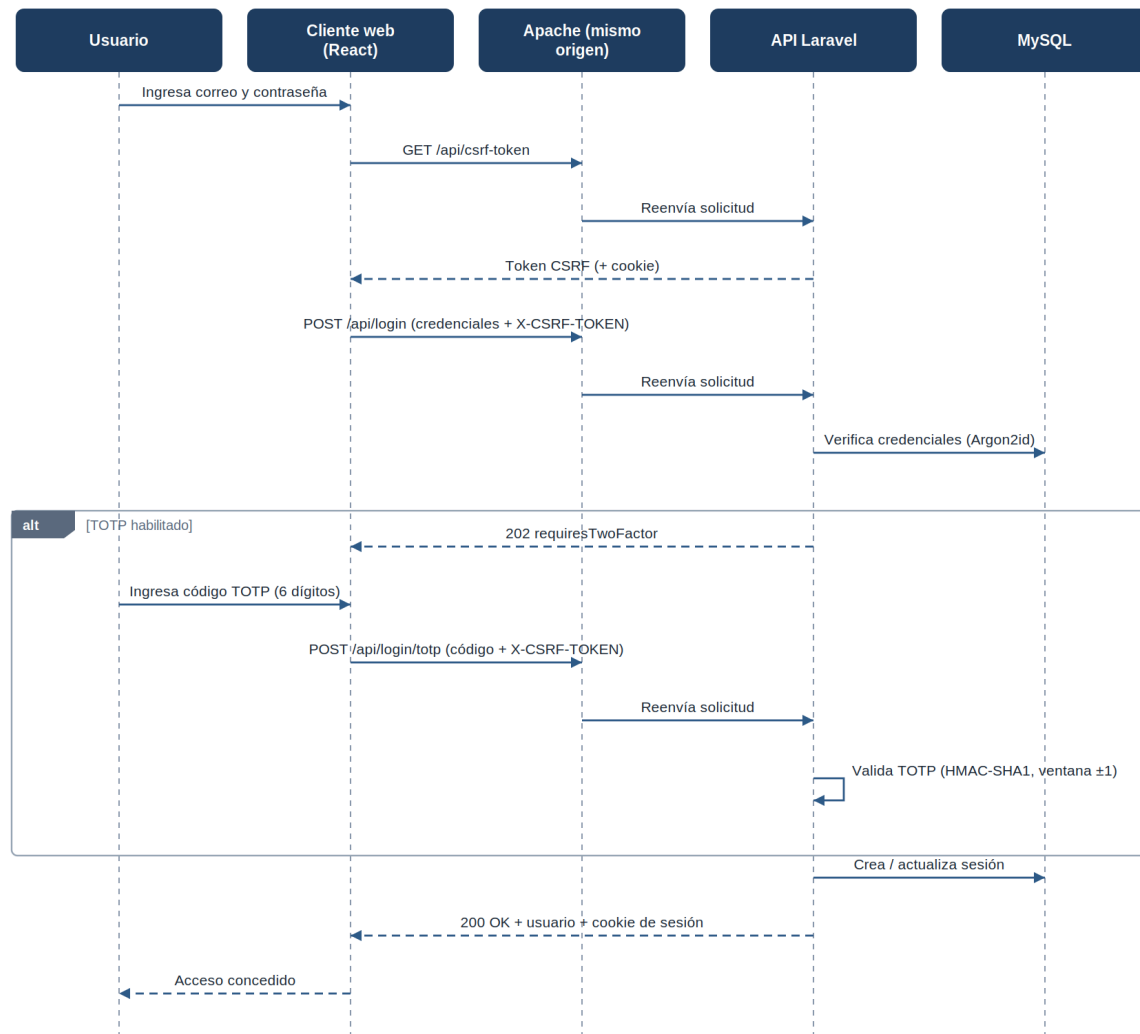


Fig. 2. Secuencia de inicio de sesión con doble factor (elaboración propia).

Gestión y verificación de documentos

Sobre el gestor de identidades de la Etapa 1, la Etapa 2 implementa el manejo seguro del ciclo de vida de los documentos confidenciales y su verificación criptográfica.

Ciclo de vida del documento.

- **Subida:** un documento (hasta 16 MB) se almacena fuera del web root público, con título, dueño inicial y nivel de confidencialidad.
- **Versionado (revisiones):** cada cambio crea una revisión que referencia a su revisión padre (cadena padre→hijo) y guarda el hash SHA-256 de su contenido, garantizando integridad y trazabilidad de los cambios.
- **Aprobación y firma:** un admin aprueba el documento; los requisitos de firma definen una secuencia ordenada de firmantes (por usuario o por rol); coordinadores y admins firman revisiones con passkeys (ver Fig. 3).

- **Borrado:** deja una "lápida" (tombstone) con metadatos, sin el contenido.

Verificación de documentos (tres niveles). El sistema permite comprobar que un documento no fue alterado y que sus firmas son auténticas, con creciente independencia del servidor:

1. **Estado de verificación en el servidor** (`GET /documents/{id}/verification`): el backend recalcula el hash de la revisión y valida las firmas registradas.
2. **Verificación local en el navegador:** el cliente descarga un verification bundle (hash, firmas y claves públicas) y, con la Web Crypto API, verifica las firmas localmente (`verifyDocumentBundleLocally`), sin necesidad de confiar en el servidor; refuerza el no-repudio y la transparencia.
3. **Paquete de verificación firmado (RSA):** el backend puede emitir un "manifiesto" del paquete firmado con RSA (RSASSA-PKCS1-v1_5 + SHA-256), con claves RSA de 2048–3072 bits, como constancia portable de integridad.

Resultados

Resultado del Backend

Se obtuvo una API REST funcional que cubre autenticación, segundo factor, gestión documental, administración y auditoría. La siguiente tabla resume los endpoints principales y el rol mínimo requerido.

Tabla 4. Endpoints principales de la API por rol (elaboración propia). La API de la plataforma organiza sus endpoints en dominios funcionales, cada uno con un nivel mínimo de autorización requerido, alineado al modelo RBAC definido previamente.

Dominio	Endpoint(s) representativo(s)	Rol mínimo
Sesión	GET /csrf-token, POST /login, POST /login/totp, POST /logout, GET /me	Público / autenticado
Passkeys	POST /webauthn/login/options·verify, POST /webauthn/register/options·verify	Público / autenticado
Enrolamiento 2FA	POST /totp/enroll/options·verify	Autenticado
Documentos (subir)	POST /documents	Voluntario+
Documentos (ver/descargar/verificar)	GET /documents, /{id}/download, /{id}/verification	No-coordinador+
Revisiones y firma	POST /documents/{id}/revisions/*, /revisions/{rev}/sign/*	Coordinador+
Invitaciones	POST /admin/invites, /{id}/verify-out-of-band, /{id}/issue-link	Coordinador+
Administración	GET /admin/users, /admin/document-approvals, /admin/signing-ledger, GET /audit-events	Admin

Resultado del Frontend

Se obtuvo una SPA funcional con pantalla de inicio de sesión (incluido login por passkey), enrolamiento de segundo factor (QR TOTP y registro de llave de seguridad), y los módulos operativos diferenciados por rol. Las siguientes capturas muestran la interfaz real en operación.

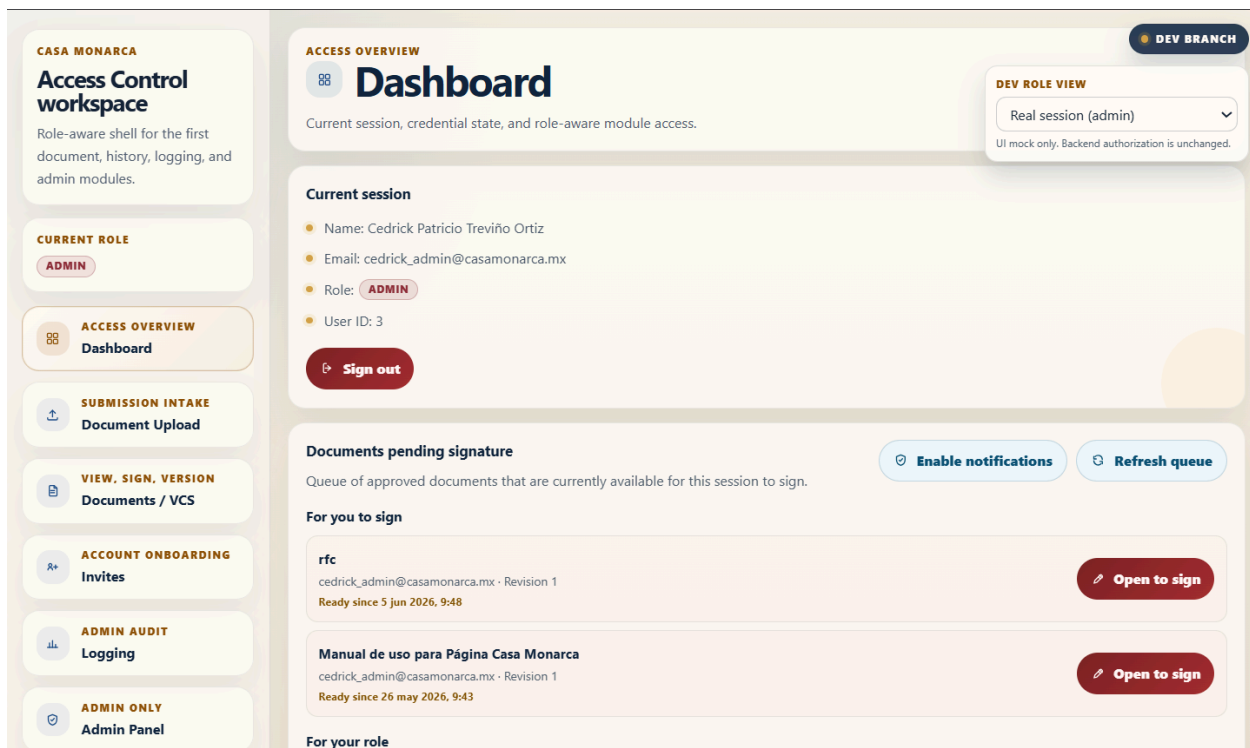


Fig. 5. Vistas reales del sistema por rol (elaboración propia).

Resultado de la solución completa

El sistema integrado se desplegó en un VPS de staging (casamonarca.muchosnumeros.online) bajo Apache, replicando las restricciones del hosting compartido objetivo. La solución cubre el ciclo completo: invitación verificada → registro → doble factor → subida y versionado de documentos → firma digital → auditoría, garantizando confidencialidad (cifrado del secreto TOTP, cookies HttpOnly), integridad (hash SHA-256 de revisiones, HMAC), autenticidad y no-repudio (passkeys) y trazabilidad (bitácora de auditoría).

El mecanismo de firma digital es el núcleo criptográfico de la solución y se ilustra a continuación.

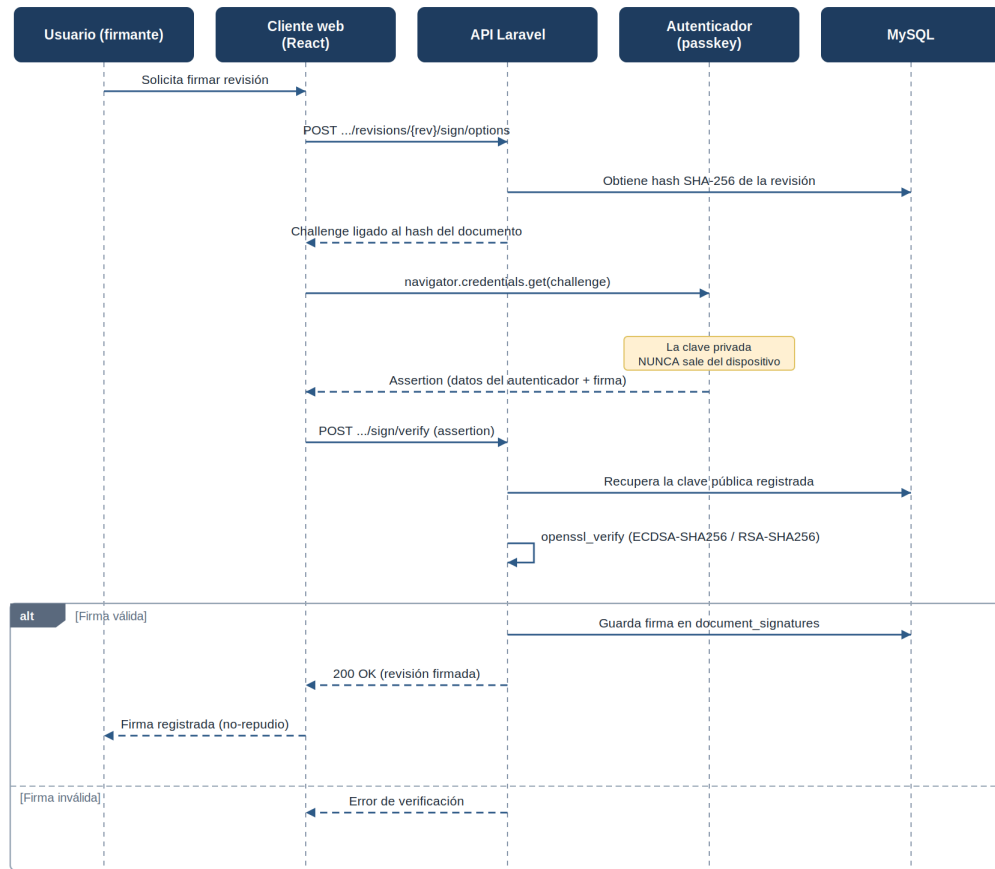


Fig. 3. Flujo de firma digital con WebAuthn/FIDO2 (elaboración propia).

Tabla 3. Inventario de algoritmos criptográficos empleados (elaboración propia). La plataforma emplea un conjunto diversificado de algoritmos criptográficos, seleccionados según la naturaleza de cada operación de seguridad.

Algoritmo	Tipo	Uso en el sistema
AES-256-CBC	Cifrado simétrico	Cifrar el secreto TOTP en la BD
Argon2id / bcrypt	Hash de contraseñas	Guardar contraseñas de forma irreversible
SHA-256	Función hash	Integridad de documentos/revisiones, hash de sesión, challenges
HMAC-SHA1	MAC con llave	Generación del código TOTP (RFC 6238/4226)
HMAC-SHA256	MAC con llave	Integridad del payload cifrado de Laravel
ECDSA P-256 (ES256)	Firma asimétrica	Firmar/verificar passkeys WebAuthn
RSA PKCS#1 v1.5 + SHA-256 (RS256)	Firma asimétrica	Passkeys RSA y firma de paquetes de verificación

Casos de uso y validaciones

Tabla 5. Casos de uso y validaciones (elaboración propia). Casos de uso críticos que fueron definidos, ejecutados y verificados para validar el comportamiento correcto de la plataforma. Todos se encuentran en estado Completado, con evidencia disponible para cada uno.

#	Caso de uso	Entrada / Acción	Resultado esperado	Estado	Evidencias
1	Alta por invitación verificada	Admin crea invitación, se verifica fuera de banda y se emite enlace	Cuenta creada con rol invitado; invitación consumida	Completado	Video [1:05 - 3:15]
2	Inicio de sesión con TOTP	Correo + contraseña + código de 6 dígitos	Acceso concedido; sesión creada	Completado	Video [3:18 - 3:45]
3	Inicio de sesión con passkey	Login con llave de seguridad/biometría	Acceso concedido por WebAuthn	Completado	Video [8:10 - 8:25]
4	Control de acceso por rol	Usuario sin permiso intenta módulo admin	Respuesta 403 forbidden_role (Visible en Logs)	Completado	Video [13:10 - 13:35]
5	Subida y versionado de documento	Subir documento y nueva revisión	Revisión con hash SHA-256 y cadena padre→hijo	Completado	Video [6:15 - 7:55]
6	Firma digital de revisión	Firmar revisión con passkey	Firma verificada con clave pública; registrada	Completado	Video [9:45 - 10:10]
7	Auditoría de acciones	Ejecutar acciones sensibles	Eventos registrados en bitácora audit_events	Completado	Video [10:44 - 12:10]

Conclusiones y trabajo futuro

- Se cumplieron los objetivos de ambas etapas: la Etapa 1 entregó un gestor de identidades y control de acceso funcional, y la Etapa 2 lo extendió con gestión, versionado, firma y verificación de documentos. En conjunto, aseguran confidencialidad, integridad, autenticidad, disponibilidad y no-repudio sobre los documentos sensibles de Casa Monarca, usando estándares abiertos y un stack sostenible para una organización sin fines de lucro.
- El enfoque de mismo origen, sesiones server-side y passkeys probó ser adecuado para el hosting compartido objetivo.
- El flujo del firmado y aprobación de documentos funciona, y cuenta con las funcionalidades necesarias para asignar orden estricto o no, y para visualizar todos los flujos por cuenta dentro de la aplicación. Además, se tienen backlogs como sistema de rastreo de actividad como evidencia de los movimientos y aprobaciones realizadas.

Trabajo futuro:

- Completar el control de versiones (VCS) de documentos: los eventos `vcs.*` están definidos pero el VCS completo quedó pendiente.
- Encadenar criptográficamente la bitácora de auditoría (actualmente append-only pero no encadenada; no es blockchain).

- Endurecer la validación de la cadena de atestación (attestation) de WebAuthn.
- Despliegue en producción en el hosting compartido definitivo y pruebas con usuarios reales.

Referencias

(Formato APA. Completar y verificar al cerrar el documento.)

- Evidian. (s. f.). Los 7 métodos de autenticación más utilizados. <https://www.evidian.com/pdf/wp-strongauth-es.pdf>
- Fortinet. (s. f.). ¿Qué son los certificados digitales? <https://www.fortinet.com/lat/resources/cyberglossary/digital-certificates>
- M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). TOTP: Time-Based One-Time Password Algorithm (RFC 6238). IETF.
- Viafirma. (2021, 12 de octubre). Métodos de autenticación. <https://www.viafirma.com/es/metodos-de-autenticacion/>
- World Wide Web Consortium (W3C). (2021). Web Authentication: An API for accessing Public Key Credentials (WebAuthn) Level 2.
- Zertuche, V. (2026, 9 de abril). ¿Qué es OTP? Todo sobre la contraseña de un solo uso. SMS Masivos. <https://www.smsmasivos.com.mx/blog/que-es-otp-codigo-de-un-solo-uso>
- RFC 8017: PKCS #1: RSA Cryptography Specifications Version 2.2. (n.d.). IETF Datatracker. <https://datatracker.ietf.org/doc/html/rfc8017>
- HostGator. (s. f.). ¿Cuáles son las compatibilidades de HostGator? HostGator Centro de Ayuda. <https://soporte.hostgator.mx/hc/es-419/articles/28439216847379-Cu%C3%A1les-son-las-compatibilidades-de-HostGator>

Anexos

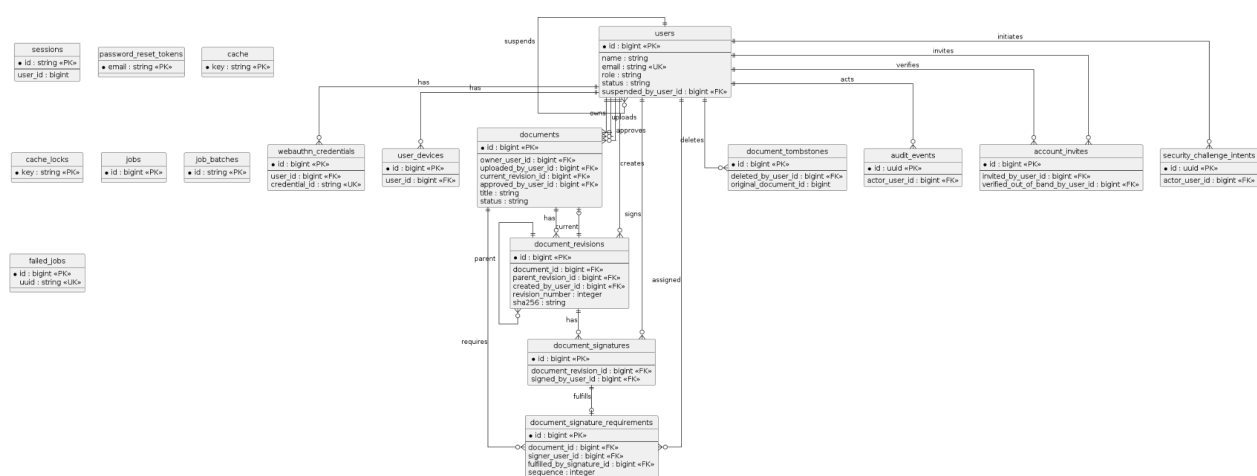


Fig. 4. Diagrama entidad-relación de la base de datos (elaboración propia).

Información complementaria que apoya las secciones de Desarrollo y Resultados:

- Diagramas Mermaid de evidencias_reto/README.md (contenedores, secuencias de login y registro, vista de despliegue, modelo conceptual).
- Esquema completo de base de datos: evidencias_reto/database-schema.md y Access-Control-CM-API/docs/architecture/.
- Link a github - https://github.com/MA2006B-601-Team-3/evidencias_reto,
<https://github.com/MA2006B-601-Team-3/Access-Control-CM-API>,
<https://github.com/MA2006B-601-Team-3/Access-Control-CM-Web>